

# Book-Finder Robot — Project Briefing

Quick recap of the architecture, the problems we identified, and the decisions we made.

## Architecture (3 components)

**Laptop (RTX 4060):** runs the main app, hosts a WiFi hotspot, does the heavy image processing (spine recognition), and shows a confidence-ranked list of matches to the user.

**Raspberry Pi 3 + camera:** mounted on top of the vertical rail. Takes photos, sends them to the laptop over WiFi, and forwards motion commands to the ESP32 over Bluetooth.

**ESP32:** drives the L298N (two DC motors for X-axis movement of the car) and the 28BYJ-48 stepper (Z-axis elevation of the camera along the mast).

## How a scan works

AprilTags are placed at the middle of each shelf row. The Pi raises the camera until it can see all N tags, records their Z positions, then descends to the lowest row. It scans right-to-left until it detects an end-of-shelf tag, then elevates to the next saved Z and scans left-to-right (serpentine pattern, saves battery). Each captured frame is sent to the laptop; the laptop returns confidence-ranked matches; the user clicks the correct one and the robot drives back to that (x, z) to point at the book.

## Problems we identified & what we decided

**Mast oscillation.** The 1.76 m aluminum mast whips when the robot starts/stops. Decision: ramp PWM up and down smoothly, lower stepper acceleration, and wait ~2 s after each move before capturing. Add a diagonal brace later if needed.

**Stepper lift (28BYJ-48).** Tested fine — lifts the Pi top-to-bottom in ~1 min and holds position when powered off. Keep an eye on slow drift during long sessions, but no change needed for the demo.

**Pi ↔ ESP32 link.** Going with Bluetooth Classic SPP (acts like a serial port on the Pi side). Pair once, bind rfcomm so it auto-connects on boot. Fallback if it gets flaky: short USB cable (both devices are on the same cart anyway).

**Pi ↔ Laptop link.** Laptop hotspot, Pi connects as client. Fallback if Windows hotspot misbehaves: a small travel router plugged into the laptop.

**Robot drifting off the shelf (X/Y problem).** Two DC motors can't go perfectly straight — over 20 m a tiny angle becomes meters of error. Decision: stick a black tape line on the floor parallel to the shelf and mount a front-facing IR line sensor array. Simple proportional correction on the ESP32 keeps us aligned. Add perpendicular tape marks at aisle start/end for absolute X reference.

**Books pushed forward by different amounts.** Realistically 2–5 cm variation. Keep the camera ~40–60 cm from the shelf so depth of field covers it. Don't pre-solve — fix only if it actually breaks during testing.

**X position tracking.** Sticking with PWM + timing for now (no wheel encoders yet). Demo-grade accuracy is enough because the photo itself shows the book; the perpendicular tape marks give us absolute corrections.

**Communication protocol.** Deferred until we have more than a couple of command types. When we get there: JSON over WiFi (laptop↔Pi), simple line protocol over Bluetooth (Pi↔ESP32), every command gets an acknowledgement.

**Spine recognition model.** Deferred. The robot has to navigate and capture stable photos reliably first. Once that works we'll have real images to train/test on.

## Build order we agreed on

- 1 Mechanical: tune soft accel + settle time, brace the mast only if needed.
- 2 Bluetooth Pi↔ESP32 link working with the current single-character test code.
- 3 Line follower: tape down on the floor, IR sensor on the front, ESP32 adjusts motors.
- 4 AprilTag Z-calibration: Pi detects tags, stores their stepper positions, can drive to any row.
- 5 End-to-end serpentine scan: capture photos at known (x, z) and upload to laptop.
- 6 Recognition + UI on the laptop side.

## Shopping list (things we still need to buy/check)

| Item                                  | Purpose                                  | Notes                                 |
|---------------------------------------|------------------------------------------|---------------------------------------|
| Black vinyl/electrical tape (~5 m)    | Line on the floor for the line follower  | 19 mm wide, matte finish              |
| IR line sensor array (3 or 5 channel) | Detect the tape, feedback to ESP32       | QTR-3A / QTR-5A or TCRT5000 module    |
| AprilTags (printed)                   | Z-row reference and end-of-shelf markers | Print on matte paper, ~5–7 cm per tag |
| USB cable Pi ↔ ESP32 (backup)         | Fallback if Bluetooth is unreliable      | Short, ~30 cm                         |
| Travel router (optional backup)       | Fallback if laptop hotspot is unreliable | GL.iNet Mango or similar (~\$20)      |
| Diagonal brace material (optional)    | Stiffen the mast if oscillation is bad   | Small aluminum extrusion + brackets   |
| Spare 28BYJ-48 / ULN2003 (optional)   | Replacement if current one fails         | Same model                            |
| Fully charged battery / spare         | Voltage drop affects DC motor speed      | Charge the morning of every test      |

*Everything else (Pi 3, ESP32, L298N, A4988, 28BYJ-48, camera, motors, mast, base car) is already on hand.*